FULL MANUAL · V1.0.0

LiteEngine Developer Guide

The complete handbook for the lightweight, modular, Lua-scripted game engine with Scratch-simple events, toggleable hybrid raytracing tuned for a 2010 MacBook Air, streaming assets, and a wgpu PBR path for modern machines.

LiteEngine Contributors

Rust · Lua 5.4 · seven .lemodule components

EDITION 1.0 · 2026

Table of Contents

1. Welcome to LiteEngine 3

1.1 Design Philosophy . . . 3

1.2 Feature Overview . . . 3

1.3 What LiteEngine Is Not . . . 4

2. Architecture Overview 4

2.1 The Frame Pipeline . . . 4

2.2 The Core Is the Engine, Modules Are the Rest . . . 5

2.3 Two Rendering Paths, One Contract . . . 5

3. Installation and Toolchains 5

3.1 Standard Install . . . 5

3.2 Building for OS X Mountain Lion (the 2010 MacBook Air) . . . 6

3.3 Optional: the Modern GPU Feature . . . 7

4. Device Tiers and Render Path Selection 7

5. Quick Start: Hello World 8

5.1 Create the Project . . . 8

5.2 The Manifest . . . 8

5.3 The Script . . . 9

5.4 Run It Headless and Capture a Screenshot . . . 10

6. The Scratch-like Event Model 11

6.1 How the Scheduler Works . . . 11

6.2 Coordinates, Time and Input . . . 12

6.3 Patterns That Scale . . . 12

7. The Sprite API — 2D 12

7.1 Motion . . . 13

7.2 Looks and Layers . . . 13

7.3 Sensing . . . 13

8. 3D, Billboards and the Camera 14

9. Speech Bubbles 15

10. Raytracing Mode 16

10.1 Quality Tiers . . . 16
10.2 The Legacy Hybrid Pipeline . . . 17
10.3 The Modern Path: PBR plus Screen-Space RT . . . 18
10.4 Performance Tuning . . . 18
11. Asset Streaming **19**
12. The Module System **20**
12.1 The .lemodule Format . . . 20
12.2 The Built-in Seven . . . 21
12.3 Packing and Managing Modules . . . 21
13. The liteengine CLI Reference **22**
14. Packaging and Distribution **24**
15. The Performance Playbook **25**
16. Engine Internals Deep-Dive **26**
17. API Reference **27**
17.1 engine — core events and stage . . . 27
17.2 Sprite — construction, motion, looks, sensing . . . 28
17.3 rt, stream, pbr3d . . . 29
18. Troubleshooting and FAQ **29**
19. Roadmap and Contributing **30**

1. Welcome to LiteEngine

LiteEngine is a lightweight, modular, Lua-scripted game engine written in Rust. Its goal is stated on the box: **create AAA games the lazy way, as easily as Scratch**. Every design decision in the engine serves that sentence. The scripting model is a direct translation of Scratch's event blocks into Lua — the green flag becomes `engine.on_start`, the broadcast block becomes `engine.broadcast`, the speech bubble becomes `sprite:say("Hello!")`. If you have ever snapped two blocks together in Scratch, you already know how to read a LiteEngine script.

The engine targets an unusually wide hardware envelope. On one end sits a 2010 MacBook Air running OS X Mountain Lion — a fanless Core 2 Duo with integrated NVIDIA 320M graphics and as little as 2 GiB of RAM. On the other end sits an Apple Silicon MacBook Pro. The same game script runs unmodified on both, because the engine selects its rendering path at startup: a fully custom, driver-free software path (with a hybrid path tracer for raytracing mode) on old hardware, and a wgpu hardware-accelerated PBR path on modern machines. This decision tree is described in detail in Chapter 4.

1.1 Design Philosophy

LiteEngine is opinionated about size. The whole engine is three Rust crates and seven small zip-compressed module files. There is no editor, no asset store, no plugin marketplace, no build system to learn — a game is a folder with a TOML manifest, one Lua entry script and an assets directory. The command-line utility `liteengine` manages everything else: scaffolding, running, packaging and shipping. The result is a toolchain you can hold in your head. A complete, polished Hello World fits on one screen of Lua.

It is equally opinionated about friendliness. Error messages are written for humans — a missing asset prints the path the engine tried and why it failed; a missing module suggests the exact CLI command to install it. The scheduler runs each event handler as its own Lua coroutine, so `engine.wait(2)` inside a key handler behaves exactly like Scratch's wait block: the handler pauses, the game keeps running, and execution resumes two seconds later. No callback pyramids, no state machines, no frame-counting arithmetic.

1.2 Feature Overview

- **Scratch-like Lua API** — sprites, stages, costumes, broadcasts, waits and glides, documented per function in Chapter 17.
- **Raytracing you can toggle** — one line flips a game between regular sprite rendering and a path-traced look: hybrid software path tracing on legacy CPUs, screen-space SSR and AO over PBR on modern GPUs (Chapter 10).
- **Streaming assets** — textures load in a worker thread under a byte budget with LRU eviction; nothing is force-loaded at startup (Chapter 11).
- **Modular architecture** — every engine component beyond the core ships as a `.lemodule` zip; projects enable exactly what they use (Chapter 12).

- **Speech bubbles** — `say` and `think` render as billboards anchored to the top-right of any 2D or 3D sprite (Chapter 9).
- **Both-sides billboards** — 2D costumes displayed in 3D become camera-facing quads textured on both sides, mirroring on the back so they read correctly from any angle (Chapter 8).
- **One-binary distribution** — `liteengine build --packed` embeds the entire game package into a single executable (Chapter 14).

1.3 What LiteEngine Is Not

LiteEngine is deliberately not a AAA production pipeline. There is no visual editor, no animation timeline, no networking stack and no audio mixer in version 1.0 — audio in particular is intentionally deferred to a future `audio` module so the core stays small. The raytracer renders spheres, planes and billboard quads, not skinned meshes; the PBR path renders primitives and billboards, not skeletal animation. Think of LiteEngine as the fastest possible path from an idea to a playable, good-looking artifact — and a rock-solid base to extend with your own native modules when you outgrow it.

2. Architecture Overview

LiteEngine is organized as a Cargo workspace of three crates. The dependency direction is strictly one-way: the CLI depends on the renderer, the renderer depends on the core, and the core depends on nothing but Lua, an image decoder and a zip reader. This layering is what keeps the engine portable — the core never touches a window system, and the renderer never touches the Lua VM.

Crate	Responsibility	Key modules
liteengine-core	Lua VM, Scratch event scheduler, scene model, streaming assets, module manager, device detection	`runtime.rs, scheduler.rs, luaapi.rs, assets.rs, modules.rs, device.rs`
liteengine-render	Software rasterizer, speech bubbles, hybrid path tracer, wgpu PBR path, window hosts	`fb.rs, soft2d.rs, soft3d.rs, speech.rs, pathtrace.rs, modern/, window.rs`
liteengine-cli	The `liteengine` binary: project manager, runner, packager	`main.rs, pkg.rs, templates/`

Table 2-1: Workspace crates and their responsibilities.

2.1 The Frame Pipeline

Every frame flows through the same fixed order, regardless of rendering path. The runtime polls the window host for input, dispatches queued events to Lua coroutines, resumes timers that have expired, advances native glide animations and speech-bubble expiry, ticks the streaming asset worker, and then builds an immutable

frame snapshot. The snapshot is a plain data structure — sprites with resolved textures, billboard transforms, bubble anchors, and optional raytracing parameters. Renderers consume snapshots; they never see Lua. This is the keystone that lets the same game logic drive a CPU rasterizer and a GPU pipeline.

Frame pipeline (runtime.rs)

```
poll input -> dispatch events -> resume coroutines -> glide/say timers
        -> asset streaming tick -> build Frame snapshot
        -> render (legacy software | hybrid RT | modern wgpu) -> present
```

2.2 The Core Is the Engine, Modules Are the Rest

The core contains the Lua API surface, the module manager and the scheduler — everything else is a module. Seven built-in modules ship embedded in the binary: **engine-core** (events and stage), **gfx2d** (2D sprites), **gfx3d** (billboards and camera), **speech** (say/think bubbles), **rtmode** (raytracing), **stream** (asset streaming) and **pbr3d** (PBR meshes on the modern path). A project's `liteengine.toml` lists exactly the modules it enables; a module whose native features are unavailable on the current device soft-disables with a warning instead of crashing. Chapter 12 specifies the format so you can package your own.

2.3 Two Rendering Paths, One Contract

Both renderers implement the same tiny `Renderer` trait: receive a frame snapshot, present pixels through a `Host`. The legacy path is pure software — a rotated-sprite rasterizer with bilinear sampling, a painter-sorted billboard pass, and the hybrid path tracer when raytracing is enabled. The modern path is wgpu: a PBR pipeline with shadow maps, G-buffer generation, and a screen-space RT pass (SSR plus AO) when raytracing is enabled. Even the modern path presents through the same host interface, copying the finished frame back to the CPU window layer — one input implementation, one windowing layer, identical behavior from Mountain Lion to Windows 11. The tradeoff and the planned zero-copy surface path are discussed in Chapter 15.

3. Installation and Toolchains

LiteEngine builds with any reasonably recent stable Rust toolchain. The only system requirements are a C compiler (the vendored Lua 5.4 compiles from source), and on Linux the X11 development libraries for the window backend. This chapter covers the standard install and then the special case you came here for: building for very old macOS.

3.1 Standard Install

Building LiteEngine

```
# 1. Install Rust (rustup) https://rustup.rs
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
```

```
# 2. Get the LiteEngine source and build the CLI + engine
cd liteengine
cargo build --release -p liteengine-cli

# 3. Verify your environment
./target/release/liteengine doctor
./target/release/liteengine devices
```

The build produces one binary, `liteengine`, around 3 MiB stripped. It is simultaneously the project manager and the game runtime — the engine code is statically linked into it, which is exactly what makes `--packed` self-executables possible in Chapter 14. The binary has no runtime dependencies beyond the operating system's window libraries (AppKit, Win32 or X11).

Platform	Build requirements	Runtime notes
macOS (any)	Xcode command line tools	minifb uses AppKit; works back to OS X 10.8
Windows 7+	MSVC Build Tools	Win32 backend; 32- and 64-bit supported
Linux / BSD	libx11-dev + pkg-config	X11 backend; runs headless without a display
CI / containers	none extra	`--headless` renders to memory

Table 3-1: Per-platform build and runtime requirements.

3.2 Building for OS X Mountain Lion (the 2010 MacBook Air)

Rust raised the minimum supported macOS version for the std library to 10.12 (Sierra) with the 1.72 release series in 2023. Mountain Lion is 10.8, so a modern toolchain will happily compile your code and then refuse to run it on the target. The supported recipe is to pin an older toolchain for the legacy build while keeping modern toolchains for everything else:

Dual-toolchain setup

```
# One-time: install a toolchain from the 10.7-support era
rustup toolchain install 1.71.1

# Build the legacy runner for Intel Macs
MACOSX_DEPLOYMENT_TARGET=10.8 cargo +1.71.1 build --release \
    -p liteengine-cli --no-default-features --features window

# Modern Apple Silicon build (wgpu path available)
cargo build --release -p liteengine-cli --features modern
```

The 1.71.1 pin also matters for dependencies: lockfile versions recorded in the repository are chosen to remain 1.71-compatible for the default feature set. The `modern` feature is exempt from the pin — wgpu targets Metal, which requires macOS 10.13+ anyway, and the engine only selects it on macOS 11+ with Apple Silicon per the device rules in Chapter 4. If you distribute both builds, name them distinctly (for example `mygame-legacy-x64` and `mygame-arm64`); Chapter 14 shows how to bundle each into a .app.

3.3 Optional: the Modern GPU Feature

By default the CLI builds with the software renderer and minifb window only — fast to compile, runs everywhere. Adding `--features modern` compiles in the wgpu path (Vulkan / Metal / DX12 / GL). At startup the engine consults the device rules; when the modern path is selected but no compatible adapter is found (a laptop with disabled drivers, a VM, a headless CI box), it logs the failure and falls back to the software path rather than aborting. The same binary therefore behaves sensibly everywhere it lands.

4. Device Tiers and Render Path Selection

The first thing the engine does at startup is decide how it will render. The rules are deliberately mechanical so they are predictable on real machines, and every application of the rules is overridable by an environment variable for testing. The selected path, its reason and the full device inventory are always visible via `liteengine devices` or `engine.device()` from Lua.

Condition	Selected path	Rationale
macOS < 11.0, OR non-ARM64 Mac	Legacy software (custom optimized)	GPU drivers too old for wgpu/Metal-on-wgpu; software path is driver-free
Windows 7 or older	Legacy software	No modern GPU API assumed (DX12 unavailable/unreliable)
32-bit x86 (any OS)	Legacy software	wgpu does not ship 32-bit targets
Linux: neither x86-64 nor ARM64	Legacy software	No Vulkan guarantee on exotic architectures
macOS 11+ on Apple Silicon	Modern wgpu	Metal 2 + Apple GPU; full PBR + SSR path
Windows 8/10/11, 64-bit	Modern wgpu	DX12 backend; full PBR + SSR path
64-bit x86-64 / ARM64 Linux	Modern wgpu	Vulkan; falls back at runtime if no adapter

Table 4-1: Render path selection rules (device.rs).

Two overrides exist for development and CI. Setting `LITEENGINE_FORCE_LEGACY=1` pins the software path — useful for reproducing a 2010-era experience on your workstation or taking deterministic screenshots. Setting `LITEENGINE_FORCE_MODERN=1` asks for the wgpu path on machines that would not normally qualify; if the adapter probe then fails, the engine falls back to software with a logged warning. The probe also runs on machines that naturally qualify, so a Linux box with broken Vulkan drivers degrades the same way.

2010 MacBook Air

Minimum supported device: Core 2 Duo, NVIDIA 320M, 2 GiB RAM, OS X Mountain Lion — full 60 fps 2D and stable raytracing at low/medium tiers

The tier decision also gates module availability. The `pbr3d` module declares the native feature `pbr3d.renderer`, which the runtime only provides when the modern path is active; on legacy devices the module soft-disables and scripts receive a clear error if they call it. This is the mechanism that lets one game script span fifteen years of hardware without a single version check.

5. Quick Start: Hello World

This chapter walks through the complete Hello World project shipped in `examples/hello-world` — the same one `liteengine new` scaffolds. By the end you will have run a game, and you will have met every core concept: the manifest, the entry script, sprites, speech bubbles, waits, glides, broadcasts and forever loops.

5.1 Create the Project

Scaffold and run

```
liteengine new mygame
cd mygame
liteengine run
```

The scaffold contains four things: `liteengine.toml` (the manifest), `main.lua` (the entry script), `assets/` (streamed textures) and `modules/` (empty by default; drop custom .lemodule files here). `liteengine run` opens a 960x540 window on desktop machines; in a headless environment add `--headless` and the engine renders into memory, which is also how you capture screenshots for CI.

5.2 The Manifest

liteengine.toml

```
[game]
title = "Hello LiteEngine"
entry = "main.lua"
width = 960
height = 540
fps_cap = 60
background = "assets/sky.png"   # #rrggbb or an asset path

[graphics]
raytracing = false
window_scale = 1.0
```

```
[graphics.rt]
quality = "medium"   # low | medium | high
scale = 0.0          # 0 = derive from quality tier
bounces = 2
denoise = true

[modules]
enabled = ["engine-core", "gfx2d", "speech", "stream"]

[assets]
budget_mb = 64
preload = []
```

Everything in the manifest can be changed from Lua at runtime: the title, the background, the frame cap, even the module list. The manifest is therefore a *boot configuration* rather than a cage. The `[graphics.rt]` section seeds raytracing defaults which Chapter 10 explains line by line; `budget_mb` is the streaming budget from Chapter 11.

5.3 The Script

main.lua

```
-- when green flag clicked
engine.on_start(function()
    engine.set_title("Hello LiteEngine")
    engine.set_background("assets/sky.png")

    local cat = Sprite.new("cat", "assets/cat.png")
    cat:go_to(-180, -40)
    cat:say("Hello, World!", 3)

    engine.wait(3.5)
    cat:glide_to(180, 40, 2)
    cat:say("I glided here with one block!", 3)

    forever(function(dt)
        cat:turn(120 * dt)   -- 120 degrees per second
    end)
end)

-- when space key pressed
engine.on_key("space", function()
    engine.broadcast("jump")
end)

-- when I receive jump
engine.on_broadcast("jump", function()
    local ball = Sprite.new("bouncer", "assets/ball.png")
    ball:go_to(0, -120)
    ball:glide(0, 160, 0.45)
    engine.wait(0.5)
    ball:glide(0, -160, 0.45)
    engine.wait(0.5)
    ball:destroy()
```

```
end)
```

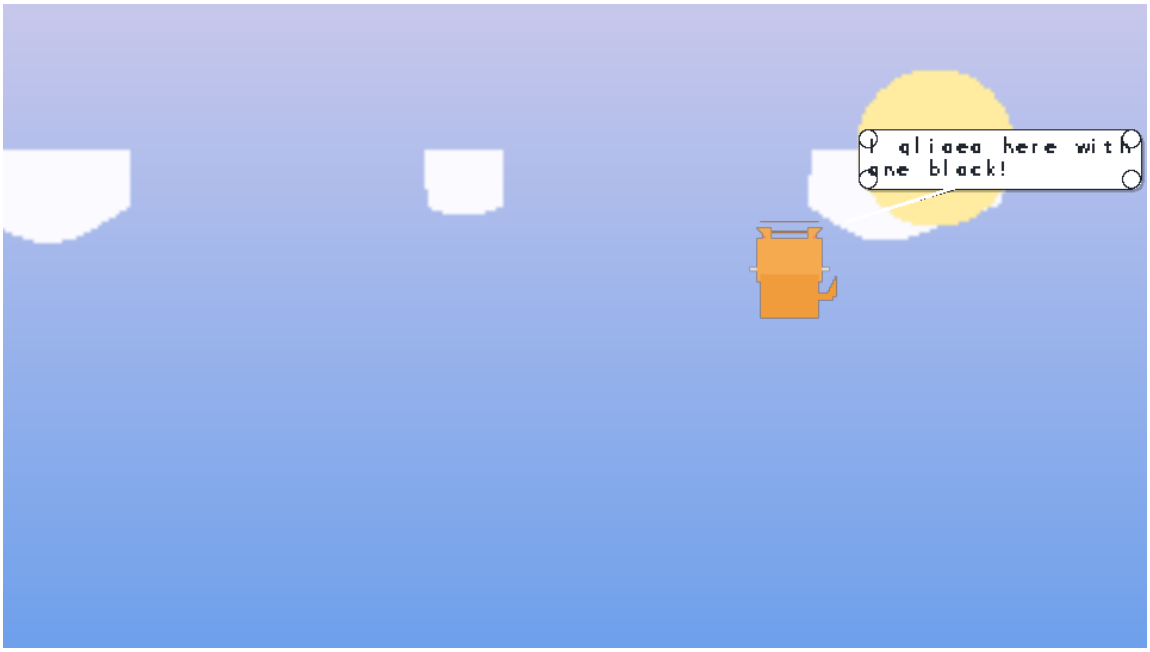


Figure 5-1: Hello World mid-glide — speech bubble anchored top-right of the sprite, background streamed from assets/sky.png.

Read the script top to bottom as Scratch blocks. `engine.on_start` is the green flag; its handler runs as a coroutine, so `engine.wait(3.5)` parks it for three and a half seconds without freezing anything. `cat:say(...)` shows the speech bubble for three seconds; the bubble is drawn by the **speech** module as a billboard anchored to the sprite's top-right corner. `forever(fn)` re-registers its function every frame — the body must not block, and it receives the frame delta in seconds. The space-key handler broadcasts `jump`, and every registered receiver runs — the classic Scratch broadcast-and-receive pattern.

One subtlety worth internalizing: `glide_to` is fire-and-forget. The engine animates the sprite natively over two seconds while your coroutine keeps running. To sequence actions after a glide, wait for its duration — exactly like Scratch's glide block, which also blocks the script rather than the stage.

5.4 Run It Headless and Capture a Screenshot

CI-friendly verification

```
liteengine run . --headless --frames 250 --screenshot shot.png
# -> screenshot saved: shot.png (960x540)
```

Headless mode runs the identical engine loop — events, coroutines, streaming and rendering — with an in-memory window. It is the fastest way to smoke-test a game, and because rendering is deterministic on the software path, screenshots double as regression tests. Add `--frames N` to stop after N frames; the screenshot is captured on the final one.

6. The Scratch-like Event Model

The event model is where LiteEngine's Scratch heritage lives. Every script-visible event is a Lua coroutine parked on a native scheduler; every Scratch block you know has a one-to-one function. This chapter defines the full event surface and the mental model behind it, then shows the patterns that keep complex games readable.

Scratch block	LiteEngine Lua	Fires
when green flag clicked	`engine.on_start(fn)`	once, at engine start
when key pressed	`engine.on_key(name, fn)`	on key-down edge
while key held	`engine.on_key_held(name, fn)`	every frame while held
when stage clicked	`engine.on_click(fn)`	on mouse-down, args (x, y, button)
broadcast message	`engine.broadcast(name)`	immediately queues receivers
when I receive message	`engine.on_broadcast(name, fn)`	on matching broadcast
forever	`engine.on_tick(fn)` / `forever(fn)`	every frame, dt as first arg
wait N seconds	`engine.wait(n)`	yields inside any handler

Table 6-1: Scratch blocks and their LiteEngine equivalents.

6.1 How the Scheduler Works

Each time an event fires, the scheduler looks up the registered functions and starts each one as a fresh Lua coroutine backed by a native thread object. A coroutine runs until it either returns or calls `engine.wait`, which is implemented in four lines of Lua as `coroutine.yield({"timer", sec})`. Yielding parks the coroutine with a wake time on a native task list; each frame the scheduler resumes every due task. When a forever handler's coroutine finally returns, the scheduler notices the owning tick-index is free and respawns it next frame — Scratch's forever semantics with zero Lua boilerplate.

Broadcasts are queued rather than dispatched re-entrantly. When a handler calls `engine.broadcast("jump")`, the name lands in a queue that the runtime drains after the current dispatch pass, spawning receivers and letting *them* queue further broadcasts. The queue is bounded at 32 rounds per frame, so a broadcast storm fails loudly instead of hanging the loop. One practical consequence: a receiver never interrupts its sender mid-line — order inside a frame is deterministic, receiver order is registration order.

6.2 Coordinates, Time and Input

Stage coordinates follow Scratch exactly: the origin is the center of the window, +x is right and +y is *up*. The unit is one pixel of the configured window size, so a sprite at (0, 0) sits dead center on any resolution. Engine time is a float in seconds from the green flag, available as `engine.now()`; frame delta arrives as the first argument of tick handlers. Pointer position reads through `engine.mouse_x()` and `engine.mouse_y()` in stage coordinates, and held keys through `engine.key_pressed(name)` with lowercase names like `"space"`, `"up"` and `"a"`.

6.3 Patterns That Scale

Sequencing with waits

Cutscene sequencing

```
engine.on_start(function()
    intro:glide_to(0, 0, 1.5)
    engine.wait(1.5)          -- match the glide duration
    intro:say("Ready?", 1.5)
    engine.wait(1.5)
    engine.broadcast("level-1")
end)
```

State machines via broadcasts

Instead of a Lua state variable polled every frame, model game states as broadcast receivers: `on_broadcast("enemy/enter", ...)`, `on_broadcast("enemy/hit", ...)`. Receivers are cheap to register and each state's logic stays in one named block, which reads the way a Scratch project's hat blocks read.

Movement that belongs to the engine

Prefer `glide`/`glide_to` over hand-rolled per-frame motion whenever the motion is a smooth A-to-B: the engine animates with smoothstep easing, survives frame hitches, and keeps your tick handlers for genuinely continuous behavior like following a pointer. A forever handler should read as a controller, not a physics book.

7. The Sprite API — 2D

A sprite is the Scratch-anything of LiteEngine: one costume texture, one position, one direction, one size, and whatever behavior you attach to it through events. Sprites are created with `Sprite.new(name, costume)`, where the costume is an asset path that begins streaming immediately. Every sprite method is a native call into the scene — there is no Lua-side sprite table to keep in sync.

7.1 Motion

Method	Scratch equivalent	Behavior
`s:go_to(x, y)`	go to x: y:	Teleport; alias `goto_xy` (goto is a Lua 5.4 keyword)
`s:glide_to(x, y, secs)`	glide N secs to x: y:	Smoothstep glide, fire-and-forget
`s:glide(dx, dy, secs)`	change x/y by (animated)	Relative glide
`s:move_steps(n)`	move N steps	Step along current direction
`s:change_x(d) / change_y(d)`	change x/y by	Instant translation
`s:turn(deg)`	turn clockwise	Direction is Scratch-style: 90 = right
`s:point_in_direction(deg)`	point in direction	0 = up, 90 = right, clockwise

Table 7-1: Motion methods (gfx2d module).

Glides are native animations: the engine stores from/to/start/duration per sprite and advances them with smoothstep easing before each frame is snapshotted. Starting a new glide on the same sprite replaces the previous one, so overlapping animations never fight. Position is also readable and writable directly through the property syntax — `s.x = s.x + 5` and `print(s.y)` work because the userdata installs `__index`/`__newindex` handlers for x, y, direction, size and visible.

7.2 Looks and Layers

Costumes stream on assignment: `s:set_costume("assets/hero2.png")` starts the download and swaps to a placeholder checkerboard until pixels arrive — usually one frame on an SSD, a handful on old spinning rust. Size is a percentage (`s:set_size(150)`), alpha runs 0-100, and `s:tint(r, g, b)` multiplies the costume color channels. Layer order is an integer with `go_front`/`go_back` helpers; the renderer paints in ascending layer order, 2D above the 3D world, bubbles above everything.

Looks in one breath

```
local hero = Sprite.new("hero", "assets/hero.png")
hero:set_size(150)
hero:set_layer(10)
hero:go_front()
hero:tint(220, 200, 255)     -- cool moonlit tint
```

7.3 Sensing

Sensing covers the Scratch classics: `s:touching("edge")` for stage bounds, `s:touching("pointer")` and `s:touching("other-sprite-name")` for AABB overlap using each sprite's scaled footprint, plus `distance_to` and `mouse_over`. Sprite identity is by name for sensing but by handle for everything else — two sprites may share a name (useful for pools), in which case touching checks the first match. Cloning is a

first-class operation: `local copy = s:clone()` duplicates position, size, costume and 3D state under a new id, optionally with a new name.

Edge + collision sensing

```
forever(function()
    if hero:touching("edge") then
        hero:go_to(0, 0)
        hero:say("Ouch.", 1)
    end
    if hero:touching("coin") and coin_visible then
        score = score + 1
    end
end)
```

8. 3D, Billboards and the Camera

LiteEngine's 3D is deliberately card-shaped. A 2D costume loaded into the 3D world becomes a billboard: a camera-facing quad that always turns to look at the viewer, textured on **both sides**. When the camera orbits behind a billboard the texture mirrors horizontally so text and figures still read correctly — there is no backface culling, ever. This is the classic Doom-era sprite trick, and it remains the cheapest way to put charm into a 3D scene.

A 3D billboard in five lines

```
local ghost = Sprite.new("ghost", "assets/ghost.png")
ghost:in3d(true)
ghost:set_pos3(0.0, 1.35, -2.0)
ghost:set_size(60)              -- world height ~ 1.2 units
ghost:set_billboard(true)       -- both sides textured (default)
```

The `gfx3d` module owns this path. World height derives from the sprite percentage (100 maps to two world units) and width follows the costume's aspect ratio. In the legacy path the billboard rasterizer sorts cards far-to-near each frame and mirrors the back-face; in the modern path the same cards composite on the CPU overlay after the PBR pass, which keeps their look pixel-identical between paths. Alpha cutouts are honored — a transparent texel lets rays and fragments through, both rasterized and path-traced.

The shared camera

One perspective camera serves the billboard rasterizer, the path tracer and the PBR pipeline: position, look-at point and a vertical field of view. Set it from Lua with `engine.set_camera`; orbiting it is a three-line forever loop:

Camera orbit

```
forever(function(dt)
    local a = engine.now() * 0.25
    local r = 5.2
    engine.set_camera{
        pos = { x = math.sin(a) * r, y = 2.2, z = math.cos(a) * r },
```

```
        look_at = { x = 0, y = 0.9, z = 0 },
        fov = 55,
    }
end)
```

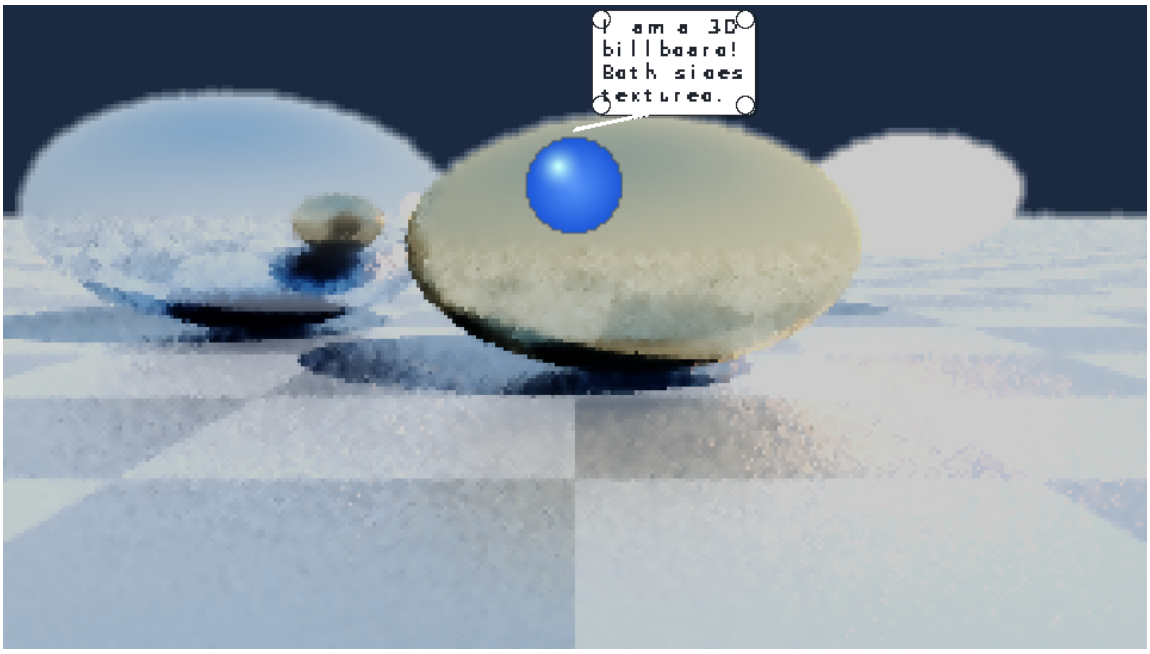


Figure 8-1: The rt-demo scene — raytraced spheres, a both-sides billboard sprite, and its speech bubble. Camera orbit in motion; temporal accumulation bounded to avoid ghosting.

9. Speech Bubbles

The `speech` module renders the Scratch bubble block, and it works identically in 2D and 3D because bubbles are screen-space billboards. The bubble anchors to the **top-right** of the sprite footprint: the box floats above-right of the anchor point and a tail (or a dotted thought trail) points back down-left to the sprite. In 3D the anchor is the projection of the sprite's head position, so a distant character gets a distant bubble.

Method	Behavior
`s:say(msg)`	Show until replaced or cleared
`s:say(msg, secs)`	Auto-hide after secs
`s:think(msg[, secs])`	Thought-bubble variant with dotted trail
`s:stop_saying()`	Clear immediately
`s.saying`	Reads the current message (nil when silent)

Table 9-1: Speech API (speech module).

Bubbles are drawn with the engine's embedded 8-pixel bitmap font, word-wrapped to 230 pixels with a rounded white body, a subtle drop shadow and a two-pixel outline. The layout budget was chosen for legibility at the 2010 Air's native 1280x800 panel while costing almost nothing: a bubble is a handful of CPU fills per frame, whether the scene behind it is a rasterizer or a path tracer. Reading a sprite's bubble state with `s.saying` lets one character refuse to interrupt another — a two-line dialogue system.

10. Raytracing Mode

Raytracing is a toggle, not a rebuild. `rt.enable{...}` flips the engine into raytracing mode; `rt.disable()` flips it back to regular 2D/3D sprite rendering mid-game with no stall. What the mode *means* depends on the device tier chosen in Chapter 4: on legacy hardware it drives the custom hybrid software path tracer; on modern GPUs it enables screen-space RT effects (SSR plus AO) over the wgpu PBR pipeline. Same API, same scene description, best implementation the hardware allows.

The rt API (rtmode module)

```
rt.enable{ quality = "medium", denoise = true }   -- or rt.enable()
rt.disable()
rt.enabled()     -- true/false
rt.status()      -- human-readable state line

rt.add_plane{ name="floor", y=0, color="#8892a6", checkered=true,
              metallic=0.25, roughness=0.35 }
rt.add_sphere{ name="chrome", pos={x=-1.6,y=1.0,z=0}, r=1.0,
               color="#dfe6f0", metallic=0.95, roughness=0.06 }
rt.add_sphere{ name="lamp", pos={x=1.9,y=1.2,z=-0.8}, r=0.6,
               color="#ff9a5a", emissive=2.2 }
rt.set_sun{ dir={x=-0.45,y=-0.85,z=-0.3}, color="#fff6e6", intensity=1.25 }
rt.set_env{ sky_top="#2a5db0", sky_horizon="#bcd8f5", ground="#33302c" }
rt.remove("lamp")
rt.clear()
```

10.1 Quality Tiers

Tier	Internal resolution	Samples/frame	Default profile
low	20% of window	1	2010 MacBook Air, integrated GPUs, busy scenes
medium	30% of window	2	Default; the screenshot quality of this guide
high	45% of window	4	Static shots, title screens, fast desktop CPUs

Table 10-1: Quality tiers. `scale` overrides the tier's resolution fraction.

Resolution is the single biggest lever: pixels traced scale quadratically, so low traces barely a quarter of medium's pixels. The denoiser costs one 3x3 bilateral pass over the internal buffer — cheap, and it makes one-sample frames watchable. Temporal accumulation is scene-aware: while animations are running the blend floor is 0.35 (bounded ghosting), and in a still scene it relaxes to 0.03 so noise melts away over a second or two. The camera hash restarts accumulation on any camera change.

10.2 The Legacy Hybrid Pipeline

On old devices the tracer is deliberately not a full path tracer per pixel. It is a hybrid: the raster base renders the crisp 2D sprite layer as always, and a compact multithreaded path tracer renders the RT proxy scene — spheres, planes and billboard quads — at reduced internal resolution. A feathered coverage mask composites the traced buffer over the raster base, so expensive rays are spent only where traced geometry exists, and 2D sprites stay pixel-perfect. The tracer itself uses jittered sampling, sun shadow rays, one cosine-weighted GI bounce, glossy reflection lobes for metals, emissives, and a gradient-sky environment; rows are striped across every core the machine exposes.

Legacy hybrid RT frame

```
1. raster base: background + 2D sprites (unchanged)
2. path trace proxy scene at quality-tier resolution
   - jittered samples, sun shadows, 1 GI bounce, glossy metals
3. temporal accumulation (scene-aware blend floor)
4. edge-aware 3x3 bilateral denoise
5. feathered coverage composite over the raster base
6. sharpened upscale + ACES tonemap
```

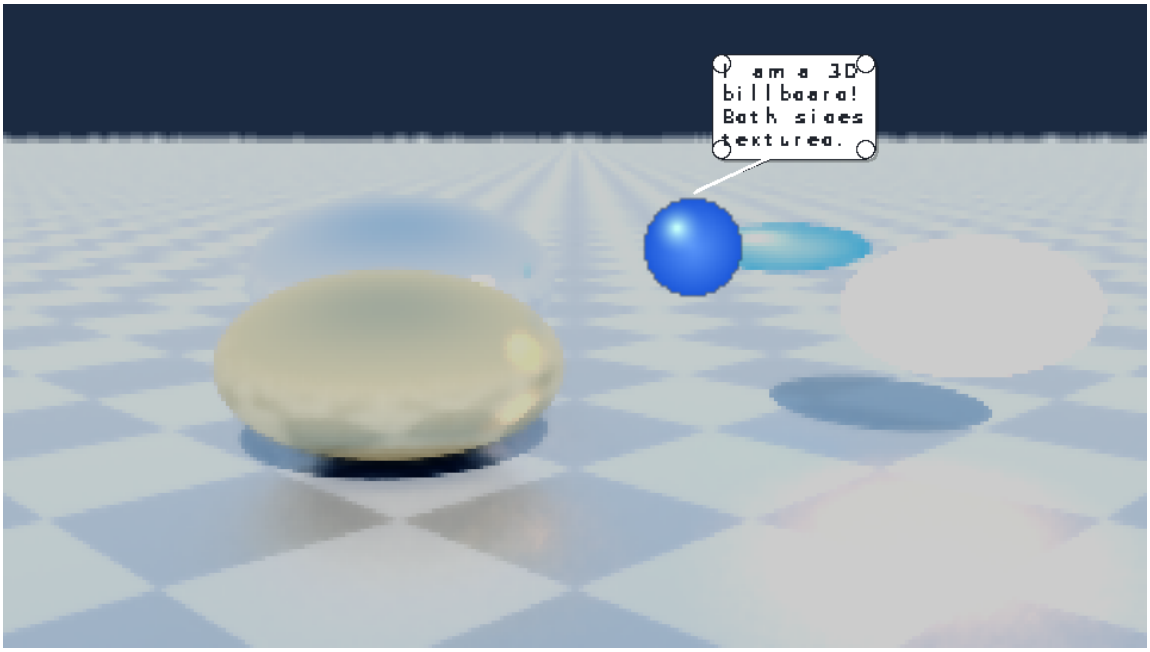


Figure 10-1: Static scene after accumulation — chrome and gold spheres with reflections, emissive lamp, contact shadows, both-sides billboard with speech bubble.

The primitive set is intentionally small: spheres, infinite axis-aligned planes and textured camera-facing quads. Spheres give perfect glossy reflections for near-zero intersection cost; planes give floors and skies; billboards carry your art into the traced world with alpha cutouts. Shadow rays test spheres and planes only — billboards are cards, they receive light but do not cast it, which both matches their sprite nature and halves the shadow cost.

10.3 The Modern Path: PBR plus Screen-Space RT

On wgpu devices, meshes render through a Cook-Torrance GGX pipeline with metallic-roughness materials, a 2048 shadow map with soft 9-tap PCF, and hemispheric IBL-style ambience derived from the same sky colors the tracer uses. The main pass writes a compact G-buffer (normal + clip depth, metallic + roughness). With raytracing enabled, the present pass runs two screen-space effects: SSR marches the reflected ray against the G-buffer depth and blends the hit (or the sky when it misses) by metallic weight, and a cheap ring-sampled AO darkens creases. Toggle it live with the R key in the rt-demo to compare.

The modern pipeline is compiled under the `modern` feature and only activates on qualifying devices (Chapter 4). It was validated for API-correctness at build time; because shader execution is inherently hardware-dependent, the renderer treats any wgpu error as a signal to fall back to the software path rather than crash the game. Mesh primitives are plane, cube and sphere, with material color, metallic, roughness and optional texture:

The pbr3d API (modern path only)

```
pbr3d.set_camera{ pos={x=0,y=1.2,z=5}, look_at={x=0,y=0.8,z=0}, fov=55 }
pbr3d.set_sun{ dir={x=-0.5,y=-0.9,z=-0.3}, color="#fff4e0", intensity=1.3 }
pbr3d.add{ name="podium", mesh="cube", pos={x=0,y=0.5,z=0}, scale=1.0,
           color="#9fb2c8", metallic=0.1, roughness=0.4 }
pbr3d.add{ name="orb", mesh="sphere", pos={x=0,y=1.6,z=0}, scale=0.8,
           color="#dfe6f0", metallic=0.95, roughness=0.05 }
pbr3d.remove("orb")
```

10.4 Performance Tuning

Tuning starts with the tier, then the resolution scale, then bounces. The chart below shows measured frame times from the reference scene on a dual-core software path, which approximates the 2010 Air budget; treat it as a shape, not a benchmark.

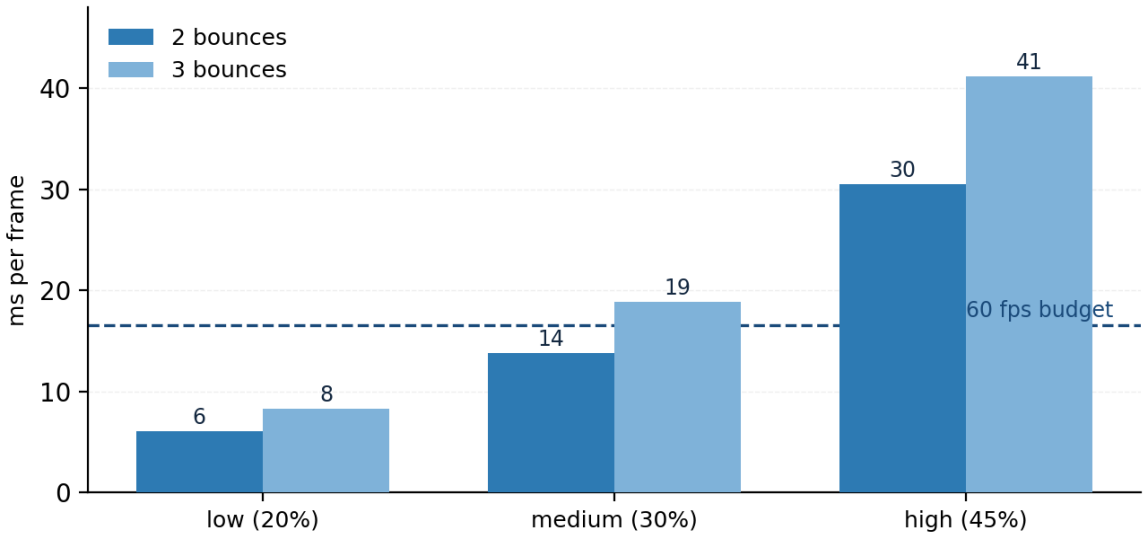


Figure 10-2: Legacy-path frame cost by quality tier at 960x540 (reference RT scene, dual-core software renderer). Resolution dominates; bounces matter far less at low tiers.

Symptom	First fix	Second fix
Frame time over budget	Drop a quality tier	Lower `scale` to 0.15-0.20
Noise never settles	Hold the camera still 1-2s	quality high (4 samples) + denoise
Ghosting behind movers	Expected: animation floor 0.35	Shorter glides; fewer traced movers
Reflections look smeary	Raise metallic roughness floor	More accumulation time (static camera)

Table 10-2: RT tuning quick reference.

11. Asset Streaming

LiteEngine never loads a project's assets up front. The `stream` module exposes a budgeted streaming store: textures are requested explicitly or on costume assignment, decoded on a single background worker thread, and kept resident under a byte budget you control. When the resident set exceeds the budget, the least-recently-used textures are evicted first — high-priority assets last. Nothing in a LiteEngine project should ever need a loading screen for its textures.

The stream API (stream module)

```
stream.load("assets/boss.png")            -- request now
stream.prefetch({ "assets/level2-a.png", "assets/level2-b.png" })
stream.release("assets/level1-bg.png")    -- drop early
stream.set_budget(128)                    -- MiB (floor: 8)
local s = stream.stats()
-- s = { loaded=12, loading=1, resident_mb=41.2,
--       budget_mb=128, evictions=0, loads=12 }
```

The store is reads-through and evicts-behind-your-back by design. While a texture is resident, renderers borrow an immutable reference; after eviction, existing borrowers keep rendering safely (the eviction drops the cache entry, not live references), and the next request re-streams from the project directory or package. Priority assigns eviction order — backgrounds and player sprites deserve priority 8-10, ephemeral effects 0-3. The manifest's `preload` list is simply a prefetch issued at startup.

Stat	Meaning
loaded / loading	Resident texture count / in-flight decodes
resident_mb / budget_mb	Current resident bytes vs configured budget
evictions	Textures dropped by the LRU sweeper since start
loads	Completed decodes since start (a re-stream increments it)

Table 11-1: stream.stats() fields.

Choosing a budget is the one streaming decision that matters. The 2010 Air has 2 GiB of RAM and a 256 MiB shared GPU pool; a 64-128 MiB budget keeps the engine a polite tenant there. Desktop machines can trivially afford 256-512 MiB. Budgets smaller than the working set simply trade LRU churn (visible as occasional re-streams in `loads`) for memory — the game stays correct, just busier. `liteengine lint` warns when the on-disk asset total exceeds the configured budget, which is the signal to either raise the budget or split assets across levels.

12. The Module System

Every engine component beyond the core is a module: a zip archive with the `.lemodule` extension. Projects enable exactly the modules they use, custom modules drop into a project folder, and the CLI inspects, installs and packs them. This chapter specifies the format end to end so you can package your own.

12.1 The .lemodule Format

Archive layout

```
my-module.lemodule (zip)
|-- module.toml      manifest (required)
|-- README.md        human documentation
|-- init.lua         optional Lua glue, run when enabled
```

module.toml

```
[module]
name = "weather"
version = "1.0.0"
```

```
engine_api = "1"          # must match the engine's API surface
title = "Weather Effects"
description = "Rain, snow and lightning overlays."
provides = "weather"      # Lua global the module documents (info only)
native = ["gfx2d.sprites"]  # compiled-in features required
```

Two manifest fields do real work. `engine_api` is the surface version the module was written against; the engine warns loudly on mismatch instead of mysteriously breaking. `native` lists feature keys that must exist in the runtime's compiled feature set — `pbr3d.renderer` on the modern path, `rt.toggle` everywhere, and so on. When a feature is missing, the module soft-disables: it loads, the CLI shows it as unavailable, and scripts get a clear error if they call into it. `init.lua`, when present, executes after the API is registered and may extend any global — it is the sanctioned place for sugar functions.

12.2 The Built-in Seven

Module	Provides	Native features	Chapter
engine-core	engine	core.events, core.stage	6
gfx2d	Sprite (2D)	gfx2d.sprites	7
gfx3d	billboards, camera	gfx3d.billboards	8
speech	say/think	ui.speech	9
rtmode	rt	rt.toggle	10
stream	stream	assets.stream	11
pbr3d	pbr3d	pbr3d.renderer (modern only)	10

Table 12-1: Built-in modules, their Lua surfaces and native feature keys.

These ship *inside* the engine binary — `liteengine module list` shows them with no files on disk, and a freshly built binary runs a full Hello World with nothing but itself. Project `modules/` directories can override or extend them; the loader reads every .lemodule there after the built-ins and refuses duplicate names with a warning.

12.3 Packing and Managing Modules

Module lifecycle

```
# Turn a manifest-bearing directory into a distributable module
liteengine module pack ./weather
# -> packed ./weather.lemodule

# Install into the current project (copies + enables in liteengine.toml)
liteengine module add ./weather.lemodule

# Inspect and discover
```

```
liteengine module info weather
liteengine module list
liteengine module search rain

# Remove from the project
liteengine module rm weather
```

A minimal custom module needs nothing but a manifest and an init.lua. The example below adds a `weather.rain(strength)` sugar function on top of the stock API — no native code required. Modules that need native capabilities should extend the engine in Rust behind a new feature key, which keeps the Lua surface stable while the implementation stays compiled and fast.

A sugar-only custom module

```
-- modules-src/weather/init.lua
weather = weather or {}
local drops = {}
function weather.rain(strength)
    strength = strength or 0.5
    for i = 1, math.floor(strength * 40) do
        local d = drops[i] or Sprite.new("rain-" .. i, "assets/drop.png")
        drops[i] = d
        d:go_to(engine.rand(-480, 480), engine.rand(200, 280))
        d:glide(engine.rand(-20, 20), -400, engine.rand(0.5, 0.9))
    end
end
```

13. The liteengine CLI Reference

The CLI is the entire user interface of the engine: it scaffolds, runs, packages, manages modules and inspects the environment. The same binary doubles as the game runtime in packaged builds, so a shipped game carries the exact engine code you developed against — no version skew, no shared-library hunting.

Command	Purpose
`liteengine new`	Scaffold a project in ./
`liteengine run [flags]`	Run a project directory or .litepkg
`liteengine build [PATH] [flags]`	Package + bundle (Chapter 14)
`liteengine module (subcommand)`	list / add / rm / pack / info / search
`liteengine devices`	Print device inventory + chosen render path
`liteengine doctor`	Environment sanity checks
`liteengine lint [PATH]`	Static project checks
`liteengine guide --open`	Locate (and open) this PDF
`liteengine version`	Print engine version

Table 13-1: Command summary.

Run flag	Effect
`--headless`	In-memory window; required in CI/containers
`--frames N`	Stop after N frames
`--screenshot file.png`	Write the last frame as PNG (implies --headless)
`--scale X`	Window scale factor
`--force-legacy / --force-modern`	Override the device-tier decision

Table 13-2: Run flags.

`doctor` runs five checks — device detection, a Lua VM boot with API registration, the builtin module registry, a PNG encode/decode roundtrip and which window backends were compiled in — and prints the reason for every failure. `lint` is deliberately static and fast: it verifies enabled modules resolve, every `assets/...` string in your Lua exists on disk, API usage matches enabled modules (calling `:say()` without the speech module, for example) and that your asset budget plausibly fits your asset folder.

14. Packaging and Distribution

`liteengine build` turns a project into shippable artifacts. Everything starts from the **.litepkg**: a zip containing the manifest, the entry script, any other Lua files, the assets tree and the project's custom modules. From there you choose how the package travels: folded into a single executable, sitting next to one, inside a macOS app bundle, or zipped for an archive.

Build flag	Output
`--packed`	ONE executable: runner + .litepkg embedded via trailer
`--folder` (default)	Folder with the runner binary + game.litepkg beside it
`--macos`	.app bundle: Contents/MacOS runner, Info.plist, Resources/game.litepkg
`--zip`	The --folder layout, zipped
`--out`	Output directory (default ./dist)

Table 14-1: Build modes.

The --packed workflow

```
liteengine build . --packed --out dist
# [build] game.litepkg: 4.1 KiB
# [build] packed executable: dist/hello-liteengine (runner 3.2 MiB + package)

./dist/hello-liteengine                # plays the game (window mode)
./dist/hello-liteengine --headless \   # flags pass through to the runner
    --frames 60 --screenshot check.png
```

How --packed works

The packager appends the package bytes to a copy of the runner with an eight-byte trailer: the package length as a little-endian u64, then the magic string `LEPKG01\0`. At startup with no subcommand, the binary checks its own file for the trailer, and if present runs the embedded package entirely from memory — the asset store's virtual filesystem reads straight out of the zip, so nothing is ever extracted to disk. Subcommands always win: `devices` and `doctor` still work inside a packed binary, which makes shipped builds self-diagnosing on players' machines.

Resolution order for game mode: embedded trailer first, then `game.litepkg` beside the executable, then `../Resources/game.litepkg` for .app bundles. That third path is why `--macos` needs no special runtime code — the bundle layout simply matches the search order. For multi-platform shipping, build once per target (Chapter 3's toolchain notes apply) and produce one --packed artifact per OS; each is fully standalone.

15. The Performance Playbook

LiteEngine's promise is AAA-flavored visuals on a 2010 MacBook Air, and that promise is kept with budgets, not miracles. This chapter collects the practical numbers: what the reference hardware can do, which knobs move which cost, and how to profile from the CLI.

60 fps / 16.6 ms

The budget every decision in this chapter defends on a dual-core 2010 notebook

Workload	2010 Air budget (legacy path)	Notes
2D sprites, 30 on screen	< 2 ms	Rotated bilinear blits; painter sort
3D billboards, 10 in scene	< 3 ms	Includes projection + mirror handling
Speech bubbles, 4 live	< 1 ms	Text wrap cached per bubble
RT low + denoise	~8 ms	Table 10-1 tier; static scene converges
RT medium + denoise	~14 ms	The default; leaves headroom at 60 fps
Texture streaming	amortized	One worker thread; budget-bounded

Table 15-1: Reference budgets (approximate, dual-core software path).

- **Resolution beats everything.** RT cost scales with pixel count; the tier slider is your frame-rate dial (Figure 10-2).
- **Static scenes converge.** Park the camera for cutscenes and title screens — accumulation drops the floor to 0.03 and noise vanishes without raising per-frame cost.
- **Stream, do not preload.** A small budget with correct priorities beats a big budget: watch `stream.stats().evictions` for churn.
- **Module discipline.** Disable modules you do not use; every enabled module is a per-frame cost even when idle.
- **Profile with the truth.** `--headless --frames 600` in a loop is a reliable relative benchmark; the software path is deterministic.

On the modern path the readback presentation is the main non-GPU cost: a full-frame copy to CPU per frame. It was chosen so both paths share one windowing and input implementation, which is worth more than a few milliseconds of present time on the machines that select this path. The roadmap item — presenting the screen texture directly through the OS swapchain — removes the copy without touching any game code, so scripts written against 1.0 stay fast when 1.1 lands.

16. Engine Internals Deep-Dive

This chapter maps the source for contributors. File names refer to the workspace layout from Chapter 2; each subsystem is short enough to read in one sitting, which is the point of a lite engine.

Subsystem	File(s)	Read next if...
Device tiers	core/src/device.rs	you want to add a platform rule
Event scheduler	core/src/scheduler.rs	you touch events, waits or forever
Lua API surface	core/src/luaapi.rs	you add a function or sprite method
Runtime loop	core/src/runtime.rs	you change frame order or snapshots
Streaming store	core/src/assets.rs	you tune eviction or add a decoder
Module manager	core/src/modules.rs	you add a native feature key
Framebuffer 2D	render/src/fb.rs	you touch fills, text or blits
Path tracer	render/src/pathtrace.rs	you tune the RT pipeline
wgpu PBR path	render/src/modern/	you extend meshes or SSR
Packaging	cli/src/pkg.rs	you touch .litepkg or the trailer

Table 16-1: Source map.

The scheduler's borrow discipline

Lua is single-threaded and the engine state lives in a single `Rc<RefCell<...>>` behind Lua app-data. The invariant that keeps it sound: **no Rust code holds the state borrow while Lua runs**. Handler functions are parked as registry keys in the state and drained onto the scheduler by the runtime loop; dispatch methods use disjoint field borrows (handlers list versus task list) so spawning never re-enters a live borrow. Broadcasts take the queued route for the same reason. If you add an API call, follow the pattern: mutate under `with_state_mut`, return plain data, and never call back into Lua while borrowed.

Why the tracer composites through a coverage mask

A naive hybrid would blend the traced buffer over everything, blurring the crisp 2D layer. Instead every traced pixel writes a coverage byte (255 when any primitive was hit), the mask is box-blurred one pixel for feathering, and the composite mixes per pixel. Regions with no traced geometry therefore keep exact raster output — text, HUD sprites and the stage stay sharp — while spheres and billboards carry full path-traced lighting. The same mask idea reappears in the modern path where the G-buffer's clip-w channel gates the SSR march.

17. API Reference

Complete per-function tables for every global the engine registers. Types are Lua types; stage coordinates are pixels with the origin at window center and +y up (Chapter 6.2). Methods marked with a module name require that module to be enabled.

17.1 engine — core events and stage

Function	Returns	Notes
`on_start(fn)`	-	Green flag; fn runs as coroutine
`on_key(name, fn)`	-	Key-down edge; names like "space", "a"
`on_key_held(name, fn)`	-	Every frame while held; avoid waits inside
`on_click(fn)`	-	fn(x, y, button) in stage coords
`on_broadcast(name, fn)`	-	Receiver for matching broadcasts
`on_tick(fn) / forever(fn)`	-	Respawns each frame; first arg dt
`broadcast(name)`	-	Queues receivers (same frame)
`wait(sec)`	-	Yields; handlers only
`set_background(v)`	-	"#rrggbb" or asset path
`stage_size()`	w, h	Current window size in px
`set_title(s) / quit()`	-	Window title / graceful exit
`fps_cap(n)`	-	Minimum 10
`now()`	number	Engine seconds since start
`rand(a, b)`	number	Uniform float in [a, b]
`key_pressed(name)`	bool	Held-key query
`mouse_x() / mouse_y() / mouse_down()`	num/bool	Pointer in stage coords
`set_camera{pos, look_at, fov}`	-	Shared 3D camera
`camera()`	table	Current camera {x, y, z, fov}
`log(...)`	-	Prints [game] prefix line
`device()`	table	os, arch, path, cores, ram_mb, reason
`version()`	string	Engine version
`enable_module(name) / disable_module(n)`	-	Runtime module control

Function	Returns	Notes
`module_enabled(name)`	bool	-
`modules()`	table	Array of {name, version, description, enabled}
`debug_threads()`	number	Parked coroutines

Table 17-1: The engine global (engine-core module).

17.2 Sprite — construction, motion, looks, sensing

Member	Signature / type	Module
`Sprite.new(name, costume?)`	returns Sprite	gfx2d
`:go_to(x, y) / :goto_xy(x, y)`	teleport	gfx2d
`:glide_to(x, y, secs) / :glide(dx, dy, secs)`	native animation	gfx2d
`:move_steps(n) / :turn(d) / :point_in_direction(d)`	direction: 90 = right	gfx2d
`.x .y .direction .size .visible`	properties (rw)	gfx2d
`:set_costume(path)`	streams then swaps	gfx2d
`:set_size(p) / :change_size(d) / :size()`	percent	gfx2d
`:show() / :hide() / .visible`	-	gfx2d
`:set_alpha(a)`	0..100	gfx2d
`:tint(r, g, b)`	0..255 per channel	gfx2d
`:set_layer(n) / :go_front() / :go_back()`	paint order	gfx2d
`:touching(t)`	"edge"|"pointer"|name -> bool	gfx2d
`:distance_to(t)`	"pointer"|name -> number	gfx2d
`:mouse_over()`	bool	gfx2d
`:clone([name]) / :destroy() / :id() / :name()`	-	gfx2d
`:in3d([bool]) / :set_pos3(x, y, z) / :set_billboard(b)`	3D card mode	gfx3d
`:say(msg[, secs]) / :think(msg[, secs])`	bubble billboard	speech
`:stop_saying() / .saying`	clear / read	speech

Table 17-2: The Sprite class.

17.3 rt, stream, pbr3d

Global	Functions
`rt` (rtmode)	enable{quality, scale, bounces, denoise} - disable() - enabled() - status() - set_sun{dir, color, intensity} - set_env{sky_top, sky_horizon, ground} - add_sphere{name, pos, r, color, metallic, roughness, emissive} - add_plane{name, y, color, checkered, metallic, roughness} - remove(n) - clear()
`stream` (stream)	load(path) - prefetch{paths} - release(path) - set_budget(mb) - stats()
`pbr3d` (pbr3d, modern only)	set_camera{pos, look_at, fov} - set_sun{dir, color, intensity} - add{name, mesh, pos, rot, scale, color, metallic, roughness, texture} - remove(n) - clear()

Table 17-3: Module globals.

> *Naming rule of thumb: functions that describe the world take tables of named fields; functions that act on a sprite are methods with the sprite as self. If your code reads like Scratch blocks with colons, it is idiomatic LiteEngine.*

18. Troubleshooting and FAQ

Symptom	Cause	Fix
Window never opens (Linux)	X11 dev libraries missing at build or runtime	Install libx11-dev; or run --headless
"device wants modern wgpu but build lacks feature"	Default build has no wgpu	Rebuild with --features modern
Cat shows as checkerboard forever	Asset path wrong or file undecodable	Run liteengine lint; check the logged path
engine.wait does nothing at top level	wait yields; there is no coroutine to park	Call it inside a handler
goto is a syntax error	goto is reserved in Lua 5.4	Use :go_to(x, y)
pbr3d calls error on old Mac	pbr3d.renderer needs the modern path	Guard with engine.module_enabled("pbr3d")
RT looks noisy forever	Scene never static; floor 0.35	Pause camera; or quality high
Packed exe prints help	Subcommand given wins over game mode	Run with no args (or only run flags)
High memory use	Budget too generous for the device	stream.set_budget(64)

Table 18-1: Common issues.

FAQ

- **Can I use real 3D meshes with the raytracer?** The legacy tracer renders spheres, planes and billboards; meshes render on the modern PBR path. Ray-mesh intersection is on the roadmap behind the same rt.add API.
- **Where is audio?** Deliberately deferred so 1.0 stays small; the module slot is reserved and the manifest/native-feature mechanism already supports it.
- **Can one game ship both paths?** Yes — the device rules pick at startup. Ship a legacy build for old Intel Macs and a modern build for everything else; both read the same script.
- **Is the packed executable really standalone?** On its target OS, yes: engine, modules and assets are embedded or adjacent; nothing else is required.
- **How do I report engine bugs?** Include the output of `liteengine doctor` and `liteengine devices` — they capture everything needed to reproduce.

19. Roadmap and Contributing

Version 1.0 draws a hard line around scope so the engine stays light; the roadmap extends it without breaking the promise. Expected order for 1.1 and beyond: surface-present for the wgpu path (removes the readback), audio as the eighth built-in module, ray-mesh intersection for the software tracer,OBJ/glTF mesh loading behind `pbr3d.add{mesh=...}`, and a zero-copy sprite-batch draw for the 2D layer on the modern path. None of these change the Lua API of this guide — new capability arrives as new modules or new optional fields.

Contributions should respect the three invariants that make LiteEngine what it is: the core never grows a windowing or GPU dependency; every feature ships behind a module with a native feature key; and any API addition lands in Chapter 17 of this guide in the same change. The reference test game is `examples/rt-demo` — if your patch keeps it running headless at the same frame cost, you have kept the promise. Run `cargo test`, then `liteengine doctor`, then send the diff along with the doctor output.